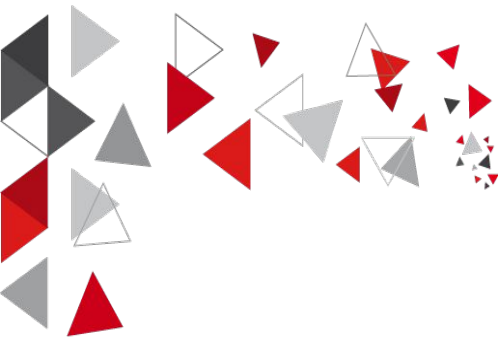
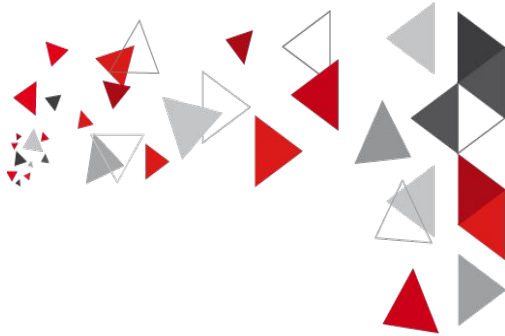


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

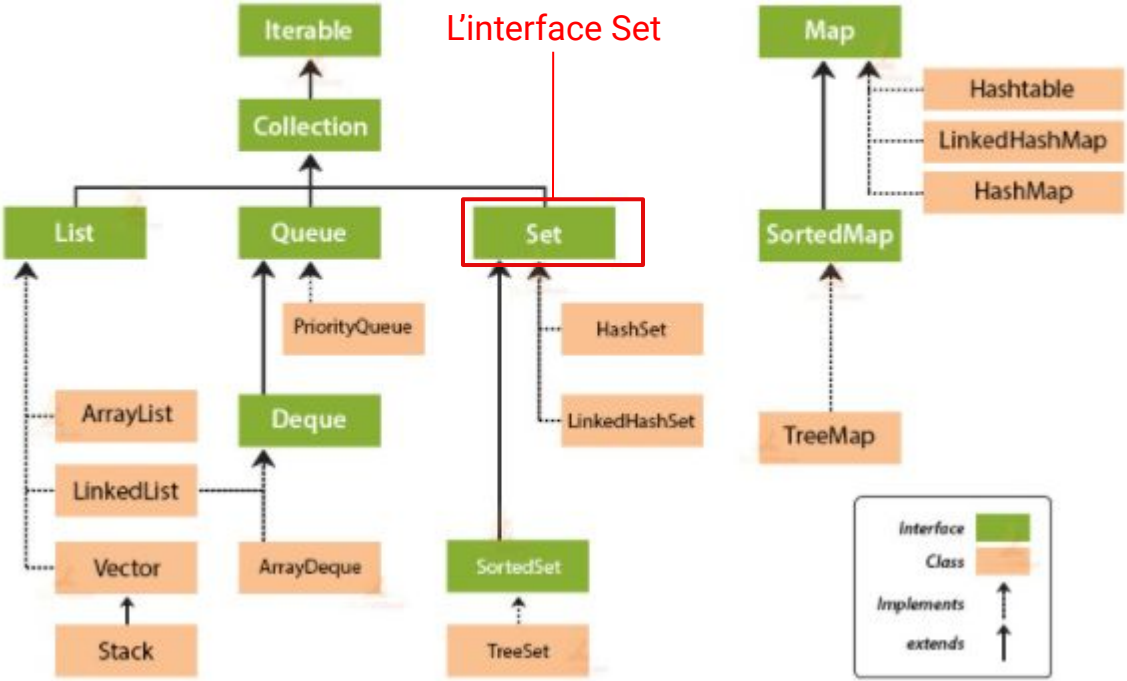
Chapitre 10 : Collection (Set)

Objectifs du chapitre

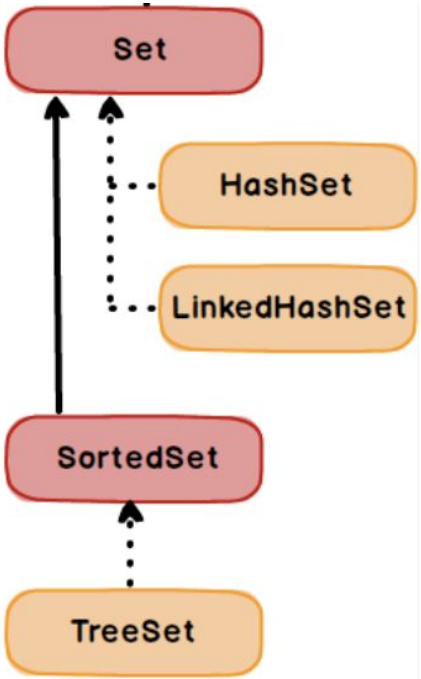


- Découvrir la Collection Set , son architecture et ses méthodes
- Comprendre la différence entre List et Set

Architecture



Set



Un **Set** représente une collection **non ordonnée** d'éléments **uniques**.

Pour les objets de la catégorie **Set**, plusieurs implémentations sont disponibles.

Les trois principales sont **HashSet, LinkedHashSet et TreeSet**.

Set



L'interface **Set** définit les caractéristiques d'une collection qui **exclut** la présence de **doublons** parmi ses éléments.

Dans une collection de type Set, les éléments ajoutés doivent implémenter les méthodes **equals() et hashCode()**. Ces méthodes sont sollicitées lors de l'ajout d'un élément pour déterminer s'il est déjà présent dans la collection.

List VS Set

Fonctionnalité	List	Set
Ordre	Les éléments sont ordonnés	Les éléments ne sont pas ordonnés
Indexation	Les éléments sont indexés, permettant un accès rapide à des positions spécifiques	Pas de support pour l'indexation directe des éléments
Doublons	Les éléments peuvent être répétés	Les éléments ne peuvent pas être répétés
Implémentations courantes	ArrayList, LinkedList, Vector	HashSet, TreeSet, LinkedHashSet
Performance pour la recherche d'éléments	Ont des performances de recherche moins efficaces car elles nécessitent souvent un parcours séquentiel des éléments jusqu'à ce que le bon élément soit trouvé.	Offre des performances de recherche rapides grâce à leur implémentation basée sur une table de hachage, permettant un accès direct aux éléments sans nécessiter de parcours séquentiel.
Performance de l'insertion / suppression d'éléments	Les Lists peuvent avoir des performances légèrement inférieures pour l'insertion ou la suppression d'éléments en plein milieu de la liste, car cela peut nécessiter de décaler les éléments suivants	Les Sets sont efficaces pour l'insertion et la suppression d'éléments individuels, grâce à leur implémentation basée sur une table de hachage qui permet des opérations rapides.

Set : Déclaration



```
Set monHashSet = new HashSet();
```

Instanciation d'un HashSet de reference Set

```
Set  monTreeSet = new TreeSet();
```

Instanciation d'un TreeSet de reference Set

TreeSet vs HashSet

Le choix entre ces deux objets est lié à la nécessité de trier les éléments :

- les éléments d'un objet HashSet ne sont pas triés : l'insertion d'un nouvel élément est rapide
- les éléments d'un objet TreeSet sont triés : l'insertion d'un nouvel élément est plus longue

Set :Exemple 1 (HashSet)

```
import java.util.HashSet;

import java.util.Set;

public class Main {

     public static void main(String[] args) {

             int count[] = {10, 20, 30, 40, 50};

             Set myHashSet = new HashSet();

             for (int i=0;i<5;i++){

                 myHashSet.add(count[i]);

             }

             System.out.println(myHashSet);

      }

}
```



La sortie:

```
[50, 20, 40, 10, 30]
```

HashSet <E>: Exemple 2

- HashSet est l'implémentation la plus utile de Set. Elle permet de stocker des objets **sans doublons.**

```java
// Création d'un HashSet de type String pour stocker des éléments uniques.
Set<String> monHashSet = new HashSet<>();

// Ajout de trois chaînes de caractères arbitraires dans le HashSet.
monHashSet.add("1");
monHashSet.add("2");
monHashSet.add("3");

// Création d'un itérateur pour parcourir les éléments du HashSet.
Iterator<String> it = monHashSet.iterator();

// Boucle while pour parcourir le HashSet tant qu'il y a des éléments suivants.
while (it.hasNext()) {
    // Affichage de l'élément suivant dans la console.
    System.out.println(it.next());
}
```

HashSet <E>: Exemple 3

```java
// Création d'un HashSet de type String pour stocker des éléments uniques.
Set<String> monHashSet = new HashSet<>();

// Ajout de trois chaînes de caractères arbitraires dans le HashSet.
monHashSet.add("1");
monHashSet.add("2");
monHashSet.add("3");

// Boucle for-each pour parcourir et afficher chaque élément du HashSet.
for (String str : monHashSet) {
    System.out.println(str); // Affichage de la chaîne courante.
}
```

HashSet <E>: Exemple 4

```java
// Création d'un HashSet de type Student pour stocker des objets étudiant uniques.
Set<Student> monHashSet = new HashSet<>();

// Ajout d'étudiants avec des identifiants uniques.
monHashSet.add(new Student(1, "Ali"));
monHashSet.add(new Student(2, "Sami"));
monHashSet.add(new Student(3, "Marwa"));
monHashSet.add(new Student(1, "Ali")); // Aucune insertion, cet étudiant existe déjà.

// La redéfinition des méthodes equals et hashCode est nécessaire pour garantir
l'unicité des objets.

// Création d'un itérateur pour parcourir les objets étudiant du HashSet.
Iterator<Student> it = monHashSet.iterator();

// Boucle while pour parcourir le HashSet tant qu'il y a des éléments suivants.
while (it.hasNext()) {
   System.out.println(it.next()); // Affichage de l'objet étudiant suivant.
}
```

HashSet <E>:

- Les éléments sont stockés dans une **table de hashage** .

- Le code de hachage (hashCode()) est utilisé pour rechercher si un objet existe déjà dans une collection. S'il n'existe aucun objet avec le même code de hachage, l'objet peut être ajouté. Si d'autres objets ont le même code de hachage, la méthode equals() est utilisée pour vérifier si l'objet est déjà présent dans la collection.

HashCode : Définition

- La méthode **hashcode()** est utilisé pour générer une valeur de code de hachage pour un objet et cette valeur de code de hachage est utilisée par certaines classes de collection pour comparer des objets, ce qui augmente les performances des grandes collections d'objets.

HashCode : Exemple

L'utilisation de nombres aléatoires (17 et 31) garantit que la valeur du code de hachage sera différente pour chaque objet.

En augmentant le nombre d'attributs inclus dans la méthode hashCode, on accroît l'unicité du nombre généré.

```
@Override
public int hashCode() {
    int result = 17;
    result = 31 * result + id;
    result = 31 * result + name.hashCode();
    return result;
}
```

La méthode hashCode() redéfinie dans la classe String

TreeSet : Définition

Le **TreeSet** stocke les éléments dans un ordre trié, assurant simultanément leur unicité. Pour utiliser un TreeSet, il est impératif que les éléments implémentent l'interface **“Comparable”** ou qu'un **”Comparator”** soit fourni. En l'absence de ces conditions, une exception de type **ClassCastException** sera déclenchée à l'exécution.

Il convient de noter que cette collection présente une performance relativement inférieure par rapport à HashSet.

Set : Exemple 1 (TreeSet)

```
Set<String> tree = new TreeSet();

tree.add("André");

tree.add("Gislain");

tree.add("Matthieu");

tree.add("Cyrille");

tree.add("Zoé");

tree.add("Thierry");

Iterator<String> it = tree.iterator();

      while (it.hasNext()){

         System.out.println(it.next());

      }
```

La classe String implémente l'interface Comparable

La sortie:

André
Cyrille
Gislain
Matthieu
Thierry
Zoé

Set : Exemple 2 (TreeSet)

```
Set<String> hash = new HashSet<>();

hash.add("André");

hash.add("Gislain");

hash.add("Matthieu");

hash.add("Cyrille");

hash.add("Zoé");

hash.add("Thierry");

System.out.println(hash);

TreeSet treeset = new TreeSet<>(hash);

System.out.println("La liste triée est: ");

System.out.println(treeset);
```

La classe String implémente l'interface Comparable

La sortie:

[Thierry, Gislain, Zoé, André, Matthieu, Cyrille]

La liste triée est:

[André, Cyrille, Gislain, Matthieu, Thierry, Zoé]

Set : Exemple 3 (TreeSet)

```
Comparator<Student> comparator = new Comparator<Student>() {

    @Override

    public int compare(Student o1, Student o2) {

       //tri croissant selon l'id

         return o1.id - o2.id;

   }};

Set<Student> tree = new TreeSet<>(comparator);

tree.add(new Student(2,"Sami"));

tree.add(new Student(3,"Marwa"));

tree.add(new Student(1,"Ali"));


System.out.println("La liste triée est: ");

System.out.println(tree);
```

La sortie:

La liste triée est:
[Student{id=1, nom='Ali'}, Student{id=2, nom='Sami'}, Student{id=3, nom='Marwa'}]

Merci pour votre attention

